AGENT TechnoStore Revenue Assistant
RUNNING USER (hypothetical grant model – permission set: TechnoStore_Revenue_Assistant2098228049_Permissions)
CHANNEL agent **FINGERPRINT** da237145496c **GENERATED** measured 2026-08-31

AGENT BLAST RADIUS REPORT

TechnoStore Revenue Assistant

Static, zero-credit analysis of the agent's real data-access surface at the execution-semantics layer.

AKSU INDEX

6

fields proven reachable beyond this user

The code executes not bounded by the running user, and these fields are ones that user may not read.

1 of those fields is labelled **regulated** by the org itself, in its own compliance group. Counted **inside** the 6 above, never added to it.

0

unproven boundaries

Real access boundaries the analysis could not prove were crossed. Reported separately and **never merged into proven** — severity is this tool's proof claim.

1

unresolved

Reach the analysis could not determine at all — dynamic SOQL, or a run context that stayed undetermined. An unknown never reads as clean.

WHAT THE AGENT REACHES VS. WHAT THE USER SEES – FIELDS

8 fields the agent's code reaches

2 of those, this user could read anyway

6 the gap between them (6 proven · 0 unproven)

Across **2** objects the agent's code touches. This report does not compute the set of objects the running user can see, so no object-level comparison is claimed here.

Aksu Index: 6 proven (1 regulated) · 0 unproven boundaries · 1 unresolved
One agent × one running user, at one moment, under one tool version – not an org score.

AKSU:1.0/P:6/C:1/B:0/U:1/fp:da237145496c
https://aksuindex.com/spec/v1.0

ACTION NEEDED

This agent's code can reach data beyond what its user is allowed to see.

We statically resolved every action discovered behind **TechnoStore Revenue Assistant** — **2** action(s), reaching **2** data object(s) and **8** field(s) — and compared that against what its user is actually allowed to see. Reach that could not be resolved is reported separately as **unresolved**, never folded into this. Nothing was run and no data left the org.

Its code can read **6** field(s) the user cannot see — **1** of them carrying a compliance label the org itself applied — and that data can reach the AI model. **This should be fixed before go-live.**

On **records**: the agent's system-mode code could read **up to 4** record(s) where this user is normally allowed **0**. That is a ceiling, not a measured result — it is a boundary to confirm; make sure those queries are limited to the right customer, not the whole table.

Every field this scan could resolve carries a classification lookup (100% of resolved fields), so the personal-data result above is a real measurement rather than a blind spot — within what was resolved. Unresolved reach is counted separately.

*The exact fixes are listed under **What to do next** below.*

SECURITY POSTURE — BY APEX API VERSION

2
APEX CLASSES ANALYSED

0
at API v67+
secure-by-default (user mode)

2
pre-v67
system-mode default — sharing/FLS bypassed by default

Pre-v67 classes: `GetRevenueSummaryAction (v63)` `SendPaymentRemindersAction (v63)`

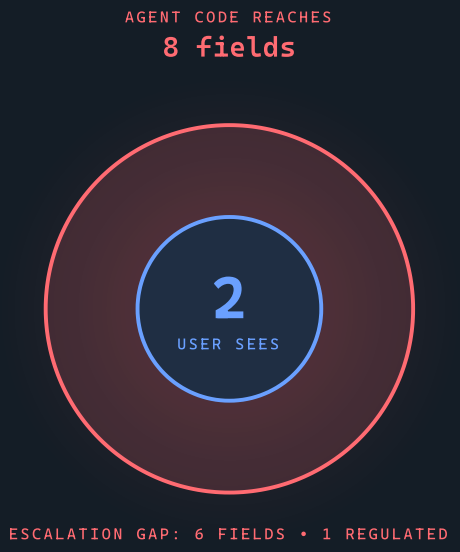
Why it matters: a class written before API v67 defaults to **system mode** — its queries and writes bypass the running user's sharing and field-level security unless the code adds an explicit `WITH USER_MODE` / `as user` clause. Upgrading the apiVersion flips that default, so orgs deliberately leave old classes as-is — and the risk is permanent until each is reviewed. Every pre-v67 class is also flagged individually (PS511).

Evidence grade: every Apex action was read from a real parse tree, so the Authority Path is traced and a finding downgraded to WARN was *proven* not to reach the model — not merely unobserved.

6

fields reachable beyond the running user — **1** regulated.

● User sees ● Agent reaches / gap



2	2	8	2/2	100%
ACTIONS	OBJECTS REACHABLE	FIELDS REACHABLE	SYSTEM-MODE	CLASSIFICATION COVERAGE

RECORD REACH — THE SYSTEM-MODE CEILING, NOT A MEASURED RESULT

The agent's **system-mode** reads could reach **up to 4** records where the running user sees **0** — an **upper-bound** record gap of **4**. (measured system-mode objects only) The whole measured gap is a **CRUD escalation** — the user has no object permission at all, so it is deterministic, not sharing-dependent.

OBJECT	READ MODE	RECORDS IN ORG	USER SEES	GAP (UPPER BOUND)	CAUSE
`Invoice`	system	4	0	≤ 4	no object permission (CRUD)

Only objects the agent's code actually **reads** are counted here (a create/insert target is a write, not a read of N records). **Records in org** is a live `COUNT()` of the whole object — it is an **upper bound, not the agent's result**: query predicates and `LIMIT` are not resolved statically. It is an escalation ceiling only for **system-mode** reads; a **user-mode** read enforces sharing, so the agent is bounded by the running user and the gap is 0 by construction. **n/a** marks record visibility that is sharing/ownership dependent and cannot be measured without running as the user — it is never estimated.

WHAT TO DO NEXT

Each item below is a concrete fix. Clear them and re-run; the report regenerates deterministically.

- [ ] `PS506` **`SendPaymentRemindersAction → Invoice.Stripe_Payment_Status__c`**

  `FIX` Remove the field from the query/output, or enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS502 SendPaymentRemindersAction → BillToContact.Email**

  FIX Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS502 SendPaymentRemindersAction → Invoice.BillToContactId**

  FIX Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS502 SendPaymentRemindersAction → Invoice.BillingAccountId**

  FIX Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS502 SendPaymentRemindersAction → Invoice.DocumentNumber**

  FIX Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS502 SendPaymentRemindersAction → Invoice.TotalAmountWithTax**

  FIX Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

- [ ] **PS503 SendPaymentRemindersAction → Invoice**

  FIX Enforce user mode (`update as user` / AccessLevel.USER_MODE) or grant the permission intentionally and document it.

- [ ] **PS503 SendPaymentRemindersAction → Invoice_Payment_Requested__e**

  FIX Enforce user mode (`publish as user` / AccessLevel.USER_MODE) or grant the permission intentionally and document it.

- [ ] **PS504 GetRevenueSummaryAction → Invoice**

  FIX Ours to close - there is nothing to change in your code. Review this operation by hand, or re-run on an analyzer build that models the shape.

- [ ] **PS509 SendPaymentRemindersAction → Invoice**

  FIX Review what 'InvoiceTrigger' writes downstream; upgrade it to API v67+ to remove the legacy default entirely.

- [ ] **PS514 SendPaymentRemindersAction (platform event: Invoice_Payment_Requested__e)**

  FIX List the subscribers of Invoice_Payment_Requested__e (Flow, process, and external) and review each as its own entry point.

- [ ] **PS516 SendPaymentRemindersAction → Invoice.TotalAmountWithTax**

  FIX Open Invoice.TotalAmountWithTax's formula and check whether any field it references is invisible to this user or carries a compliance label.

FINDINGS IN DETAIL

ERROR · 8

PS506 **SendPaymentRemindersAction → Invoice.Stripe_Payment_Status__c**

Regulated field Invoice.Stripe_Payment_Status__c is read in system mode and reaches the model, but the running user has no FLS on it.

Why ComplianceGroup GDPR. A field the running user cannot see can reach the LLM and the end user's screen. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Remove the field from the query/output, or enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS502 **SendPaymentRemindersAction → BillToContact.Email**

Field BillToContact.Email is read in system mode; the running user has no FLS on it.

Why In system mode Apex ignores FLS, so the field's data can reach the agent. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS502 **SendPaymentRemindersAction → Invoice.BillToContactId**

Field Invoice.BillToContactId is read in system mode; the running user has no FLS on it.

Why In system mode Apex ignores FLS, so the field's data can reach the agent. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS502 **SendPaymentRemindersAction → Invoice.BillingAccountId**

Field Invoice.BillingAccountId is read in system mode; the running user has no FLS on it.

Why In system mode Apex ignores FLS, so the field's data can reach the agent. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS502 **SendPaymentRemindersAction → Invoice.DocumentNumber**

Field Invoice.DocumentNumber is read in system mode; the running user has no FLS on it.

Why In system mode Apex ignores FLS, so the field's data can reach the agent. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS502 **SendPaymentRemindersAction → Invoice.TotalAmountWithTax**

Field Invoice.TotalAmountWithTax is read in system mode; the running user has no FLS on it.

Why In system mode Apex ignores FLS, so the field's data can reach the agent. Authority Path CONFIRMED: the field's value flows to the action's @InvocableVariable output, so it reaches the model.

Fix Enforce FLS (WITH USER_MODE / Security.stripInaccessible).

PS503 **SendPaymentRemindersAction → Invoice**

update on Invoice in system mode; the running user has no edit permission on this object (dml API v<=66 default).

Why In system mode Apex ignores CRUD, so the action writes an object the running user cannot - a write escalation.

Fix Enforce user mode (`update as user` / AccessLevel.USER_MODE) or grant the permission intentionally and document it.

PS503 **SendPaymentRemindersAction → Invoice_Payment_Requested__e**

publish on Invoice_Payment_Requested__e in system mode; the running user has no create permission on this object (EventBus.publish API v<=66 default).

Why In system mode Apex ignores CRUD, so the action writes an object the running user cannot - a write escalation.

Fix Enforce user mode (`publish as user` / AccessLevel.USER_MODE) or grant the permission intentionally and document it.

WARN · 4

PS504 **GetRevenueSummaryAction → Invoice**

Reach for this operation could not be fully determined - this analyzer does not model the shape (aggregate/function select - fields not enumerated).

Why A silent false-clean is worse than an honest unknown, so this is counted as unresolved rather than passed. This one is OUR limit, not a property of your code: the reach is written out in the source and analyzer build 257203d65b68 does not model this shape. Stated as of that build - a later one may resolve it, and this report is not falsified when it does.

Fix Ours to close - there is nothing to change in your code. Review this operation by hand, or re-run on an analyzer build that models the shape.

PS509 **SendPaymentRemindersAction → Invoice**

DML (update) on Invoice fires trigger 'InvoiceTrigger' at API v60 (< v67) — a legacy cascade boundary, but no escalating write was proven.

Why A pre-v67 trigger runs its DML in system mode, so this is a real boundary to review; however no DML was observed in its own body (it may delegate to a handler, or perform none), so this is flagged as a boundary rather than a proven escalation.

Fix Review what 'InvoiceTrigger' writes downstream; upgrade it to API v67+ to remove the legacy default entirely.

PS514 **SendPaymentRemindersAction (platform event: Invoice_Payment_Requested__e)**

This action publishes Invoice_Payment_Requested__e. The publish is analysed as a write; any Flow, process, or external subscriber is NOT.

Why No Apex subscriber trigger exists on it, but a platform event can also be consumed by a Flow, a process, or an off-platform subscriber, each running in its own transaction as a different user. Publishing is how an agent starts work it could not do inline, so the true blast radius can be larger than this report. An honest unknown edge, not a proven leak.

Fix List the subscribers of Invoice_Payment_Requested__e (Flow, process, and external) and review each as its own entry point.

PS516 **SendPaymentRemindersAction → Invoice.TotalAmountWithTax**

This field is a FORMULA; the fields it reads are not resolved here.

Why A formula's value is computed from other fields, so the running user's FLS on THIS field does not bound what its value carries - a formula they are allowed can echo one they are not. Unlike every other reach, this is not settled by user mode: a v67 read enforces FLS on the formula, not on its inputs. Reported as an unresolved reach, not as a proven leak - what the platform does here is not measured.

Fix Open Invoice.TotalAmountWithTax's formula and check whether any field it references is invisible to this user or carries a compliance label.

INFO · 2

PS511 **GetRevenueSummaryAction (API v63.0)**

Custom action class predates API v67 (secure-by-default).

Why Pre-v67 classes keep legacy execution semantics indefinitely until upgraded.

Fix Plan a migration to API v67 and re-review.

PS511 **SendPaymentRemindersAction (API v63.0)**

Custom action class predates API v67 (secure-by-default).

Why Pre-v67 classes keep legacy execution semantics indefinitely until upgraded.

Fix Plan a migration to API v67 and re-review.

ORG HEALTH — BEYOND THIS AGENT

Whole-org signals that don't concern **TechnoStore Revenue Assistant** directly, but anyone securing this org should know. Static Tooling-API reads — zero credits, no records returned.

> **Why this matters for TechnoStore Revenue Assistant:** an agent can only reach what the code behind it reaches. Where no explicit access mode overrides it, the Apex **API version** sets the default for every database operation. At **v67+** that default is the running user's mode, so an agent reaches *nothing its user cannot see*. **Below v67** it flips to system mode and the agent reaches *past* its user. An explicit `WITH SYSTEM_MODE` still overrides either. That is exactly the **6-field escalation** this report found above: TechnoStore Revenue Assistant's action code is pre-v67, so it reads fields its running user has no access to. Had those classes been v67, that gap would be **0** — the agent would be bounded to its user by the platform itself. This agent's own action code: **2** of 2 class(es) are pre-v67.
>
> The whole-org counts below are that same condition at scale — a map of where the *next* agent or action will escalate, before it is built.

113/113	2	1	0
YOUR APEX STILL PRE-V67 100% system-mode by default	GRANT "MODIFY ALL DATA"	GRANT "VIEW ALL DATA" ONLY	PUBLIC-OWD CUSTOM OBJECTS

Why this matters: a class or trigger below API v67 defaults to **system mode**, so its queries and writes bypass the running user's sharing and field-level security unless the code opts in — this agent uses only a couple of them, but the same latent escalation sits in every other pre-v67 file; **2** permission sets grant **Modify All Data**, which overrides all sharing and FLS for whoever holds them — an agent running as such a user has no access boundary at all; every custom object defaults to **Private** sharing — a healthy baseline.

Produced by static analysis. No agent was invoked. 0 Flex Credits. Bound to fingerprint da237145496c, which seals both the INPUTS (agent config, the analysed Apex/Flow, the permission snapshot, and what the analysis identity could see) and the TOOL that produced this verdict (analyzer 257203d65b68, parser 5.1.0; each analysed class's own apiVersion is bound per action, since it decides the verdict). Regenerate if any of these change. The Aksu Index is defined by a public specification, v1.0: aksuindex.com